

Sports Competition Framework

Modeling Refinement

This report records the gaps found in an earlier model of the system, the probes used to expose them, the design axes they produce, and the redefined model built upon them.

Authors	Aymeric Zaluski (ER2210) & Beatriz Duarte (ER2204)
Course / Instructor	Software Design and Modeling, Bartosz Walter
Date	15 June 2026

In this report

- 1. Gaps in the Current Model:** what the first submission got wrong, by the teacher and by ourselves.
- 2. Breaking the Model with Stress-Test Probes:** three disciplines that expose the hidden assumptions, and the design axes that follow.
- 3. Defining Concepts and Their Relationships:** the redefined model: entities, contracts, strategies, relationships and lifecycle.
- 4. Decision Flows, Differences and the Diagram:** the non-trivial flows, each real-world difference read as a design decision, and the class diagram.
- 5. Conclusion:** what the redesign achieved and why adding a discipline now means writing strategies, not editing the core.

1. Gaps in the Current Model

Before a single class is redrawn, the previous model must be understood on its own terms. A design cannot be corrected until it is clear exactly where, and why, it falls short. This section therefore sets the solution aside and simply catalogues the weaknesses of the first submission: both the points raised in the teacher's feedback and the deeper structural issues our own review surfaced. Only with this map of what is wrong can the redesign proceed deliberately, rather than patch the model by guesswork. They are ordered from most to least critical, with teacher-flagged issues first.

1.1 Gaps identified by the teacher

Concepts are named but not defined: Sport, Match, Competition, Participant, Result and Event are labelled but never given a definition, responsibilities or invariants. The explicitly required deliverable is absent.

Relationships between concepts are missing: No multiplicities or ownership are stated. "Participation" appears in the sequence diagrams but never in the class model, despite being the key associative concept (it carries the per-match role).

The model is flat and match rules are not modeled: The decisive difference (what ends a match and how its outcome is decided) is reduced to attributes; football's time limit and chess's terminal conditions have no representation.

Differences are described but not interpreted: The comparison table lists real-world differences but never states their consequence for the design (e.g. "numeric vs symbolic result" never leads to "therefore Result is an interface").

Use cases are out of scope: The 14 CRUD cases (create/view/modify/delete) are generic application plumbing identical for any app and say nothing about the domain model.

Low-value sequence diagrams: Most diagrams are self-evident; only the decision-bearing flows (termination, heterogeneous results into one leaderboard, pairing) are worth keeping.

1.2 Gaps that we self-identified

Head-to-head is baked in as universal: "Match = two sides producing one shared result" is treated as the base, but it is a football/chess trait, not a universal one (races and judged rankings break it).

Core/extension boundary is never argued: Which contracts are truly shared by all disciplines versus sport-specific is asserted through a hierarchy, never justified against a discipline outside the assigned two.

Scoring is misplaced on the sport: Points systems (3/1/0 vs 1/0.5/0) are attached to the sport, but they belong to the competition/format. The same sport can use different point systems in different tournaments.

Match lifecycle and states are absent: No scheduled / in-progress / finished states and no forfeit, walkover, abandonment or adjournment, all of which produce results that still affect standings.

Tournament formats are undermodeled: League, Round-Robin, Swiss and Knockout differ in fixture/pairing generation, standings aggregation and tiebreakers, none of which exist beyond a generic `computeRanking()`.

Events-vs-result link is unspecified: It is never decided whether the result is computed from events (football) or declared (chess); the two sports silently use incompatible mechanisms under one API.

Parallel inheritance smell: `FootballMatch/ChessMatch`, `FootballResult/ChessResult`, `FootballStats/ChessStats` form parallel hierarchies; Stats are stored as subclasses when they should be computed from results.

Two themes run through these gaps. The concepts were never defined and their relationships never stated; and the model silently assumed that every contest is football or chess: two sides, one shared scoreline, a fixed clock, and points owned by the sport, an assumption that races, tournaments, and judged events all break.

The correction proceeds in three parts. First, we break the model on purpose by confronting it with disciplines it was never built for, so that the failing assumptions become visible. Second, we rebuild it on precise definitions: the core entities, the contracts they must honour, the rules those contracts delegate to, and the relationships and lifecycle that bind them. Third, we draw the result as a class diagram, keeping only the sequence diagrams that carry a real decision. The next section opens the first of these three.

2. Breaking the Model with Stress-Test Probes

With the weaknesses catalogued, the most reliable way to test a design is to try to break it. A model only reveals its hidden assumptions when it meets a case it was never shaped for, so rather than reason in the abstract we add disciplines the framework was not built around and watch which parts give way. Each such discipline is a probe: not a sport we intend to deliver, only a stress test aimed at a specific assumption. Three probes proved enough to stabilise the design, each chosen to attack a different load-bearing belief.

The first probe is a race, a marathon. Football and chess both assume a contest has exactly two sides that together produce one shared result; a race has neither. Dozens of runners compete at once, and the outcome is not a score between two parties but an order of finishers. The race therefore tests the most fundamental assumption of all: that a contest is two-sided.

The second probe is tennis. Tennis is two-sided, so it leaves that first assumption intact and instead targets another: that a result is flat. A tennis match is built from sets, sets from games, games from points, each level a contest in its own right. It tests whether the model can express a result that is itself composed of smaller results.

The third probe is figure skating, a judged sport. Every discipline considered so far settles its outcome objectively: by counting, by rule, or by measurement. In skating the outcome is produced by a panel of judges. It tests whether the model can accommodate an outcome that is neither counted nor declared nor measured, but awarded.

Together the three attack cardinality, composition and the nature of the outcome: the assumptions most likely to be wrongly hard-wired. The pages that follow take each probe in turn, noting what it breaks, what it merely bends, and what it leaves standing.

2.1 First probe: Race

Breaks: Match as two sides with one result can no longer be the base: a Contest holds 1..* participations, the two-sided match becoming only a special case. Result stops being a score between two sides and becomes an ordering of participations, each with its standing-contribution. Stats are confirmed as computed aggregations, not stored subclasses.

Bends: the per-participant outcome (time, position, status) moves onto Participation; outcome determination gains a measured mode (a clock, then comparison); termination becomes two-level (a runner finishes, the contest finishes at a cutoff); the event log becomes optional; standings become pluggable per competition.

Holds: Sport and Participant, since a runner and a relay squad are both just one opaque participant.

2.2 Second probe: Tennis

Breaks: a match is a hierarchy (match, set, game, point), each level a contest with its own termination and outcome rule, so Contest must become a Composite and the rules apply at every level, *which we ultimately realise at the competition tier, not on Contest, since tennis is a probe, not a delivered module.*

Bends: outcome determination gains an aggregated mode as sub-results roll up; Result gains a composite shape; scoring may be by round reached; fixtures gain seeding.

Holds: cardinality (two sides, doubles being one participant that is a pair), the top-level Result contract, Sport and Participant.

2.3 Third probe: Figure skating

Breaks: outcome determination gains a fifth, judged mode: a panel marks each performance, a rule combines the marks into a score, and the ranking follows. None of measured, declared or counted fits.

Bends: an Official appears as an optional source of the outcome, decisive here, passive in football and chess, absent in a race; Result holds, with a constructed composite score underneath.

Holds: ranking of many sides as in the marathon, the separation of outcome from points (the judges' total is the contest outcome, not the standings points), Sport and Participant.

2.4 Resulting design axes

The probes converge on a small set of independent axes: each is one dimension along which disciplines vary, answered by exactly one design mechanism. Together they let a single configurable Contest and Competition replace the parallel football and chess hierarchies; only Sport and Participant survive every probe untouched.

Cardinality and role: from two sides with roles (football, chess) to many sides with none (a race); answered by a Contest that holds 1..* Participation, where "exactly two" is an invariant, not a subclass.

Composition: atomic in every delivered discipline: football and chess are single contests; the nesting tennis would require is treated as a probe therefore not built into Contest. The Composite pattern is instead applied one tier up, at the competition level: a **Championship** aggregates child **Standings** (single competitions, or nested championships) into one overall table, as required. Contests stay atomic.

Termination: by time, by condition, by completion-and-cutoff, or by per-level threshold; answered by a TerminationRule strategy.

Outcome determination: event-derived, declared, measured, aggregated or judged; answered by an OutcomeRule strategy with five modes.

Result representation: a scalar pair, a symbol, a ranked list or a composite; answered behind the single Result interface, an ordering plus standing-contributions.

Event recording and officials: authoritative in football, a mere log in chess, splits in a race, scorecards in skating; answered by an optional capability the result never assumes.

Scoring: 3/1/0, 1/0.5/0, position points, or points by round reached; answered by a ScoringRule owned by the competition, not the sport.

Format: league, round-robin, Swiss or seeded knockout; answered by a bundle of FixtureGenerator, StandingsAggregator and Tiebreak.

These axes sort into four kinds of mechanism: universal interfaces (Contest, Participation, Result, Competition); pluggable strategies (TerminationRule, OutcomeRule, ScoringRule, Format); optional capabilities (EventRecording, Official, roster); and no core inheritance, since a participant is opaque and the only structural pattern is Composite.

These axes are the bridge from diagnosis to design. They turn each difference the probes exposed into a concrete decision: an interface here, a strategy there, an optional capability elsewhere. What remains is to give every one of these a precise definition, with the responsibilities and rules it must always satisfy, which is the work of the next section.

3. Defining Concepts and Their Relationships

Now we turn the axes into precise definitions: each concept gets a definition plus the responsibilities and rules it must satisfy, the deliverable the first model lacked. The structure is two entities, four universal contracts, the strategy layer they defer to, and the wiring and lifecycle that connect them.

3.1 Entities

Our contracts rest on two core entities: a participant playing in a sport. Both are plain things with identity and little behaviour of their own.

A **participant** is anything that competes, either a single person or a team, with a name and an identity. The core always treats it as one opaque thing. Optionally, a roster capability exposes its members for disciplines that record per-player events or line-ups; the roster may change per contest.

Sport is the discipline itself. It works as a profile: it says which rules, which roles, and which optional capabilities a contest in this discipline uses, and which formats are typical. It does not own the points system, only the competition does.

3.2 Contracts

From the axes, four contracts are universal and every discipline honours them. Of the four, only Result is a true “many shapes” interface; Contest, Participation, and Competition are concrete classes that are configured by strategies rather than subclassed.

3.2.1 Contest

A contest is a self-contained encounter that can produce a result on its own: a football match, a chess game. (Nested contests, which tennis would need, are not part of the delivered model; multi-level structure lives at the competition tier instead, see §3.2.5.)

- **Responsibilities:** holds its participations; tracks its own state (scheduled → playing → finished); asks its termination rule when it is over and its outcome rule what the result is; produces exactly one result when finished.
- **Invariants:** at least one participation at the start (a discipline may require exactly two); a participant takes part only once in it; a result exists only once it is decided, including some endings without normal play (forfeit, walkover); abandonment yields a result only when the discipline rules it stands; it never goes back from finished to playing.
- **Boundaries:** doesn't know about points, standings, nor other contests; doesn't always assume there are only two sides; doesn't assume events decide the result.

3.2.2 Participation

Participation is the link between one participant and one contest. It holds the two things that only make sense for that pairing: the participant's role here, and how they finished. (This was the concept missing from the first model.)

- **Responsibilities:** points to its participant and its contest; carries the role / seed / lane if the discipline has one; carries the outcome: a status (entered, finished, did-not-finish, did-not-start, disqualified, withdrew) and a value (goals, time, points...) once known.
- **Invariants:** exactly one participant and one contest; its status matches the contest's state (it cannot be "finished" while the contest is only "scheduled"); no two participations in the same contest share the same participant; a role, if used, comes from the discipline's allowed set (e.g. one home and one away).
- **Boundaries:** doesn't work out its own points (that happens later, at the competition); doesn't assume it has an opponent, for example in a race it just has a time and a place.

3.2.3 Result

Result is what a finished contest produces. Its one job: give an ordering of the contest's participations and, for each, what it contributes to the standings. A 2–1 score, a win/draw/loss, a list of finishing times, a set-by-set score are all just different shapes of this.

- **Responsibilities:** ranks the participations (ties allowed); for each, exposes a simple standing-contribution ("won", "drew", "3rd", "0.5") the competition can turn into points without knowing the sport.
- **Invariants:** it exists only once the contest is finished and does not change afterwards (a correction means deliberately re-opening the contest, not a quiet edit); everyone who took part appears exactly once, including those who did not finish or were disqualified (ranked last or flagged, never dropped); ties are allowed.
- **Boundaries:** doesn't hand out points, doesn't know the points system, nor know how the outcome was reached (counted, declared, measured, judged). It only carries the final ordering.

3.2.4 Competition

A competition is a group of participants playing under a format, with standings worked out by a scoring rule over the contests' results. It is where fixtures are created, points are given, and a leaderboard appears. For example, one marathon is a competition with a single contest; a league is one with many.

- **Responsibilities:** holds its entrants and its contests; owns a format (creates fixtures, builds standings, breaks ties) and a scoring rule; asks the format to generate contests, sometimes round by round (Swiss, knockout); turns each finished result into points; produces the leaderboard on demand.
- **Invariants:** every contest uses only the competition's entrants; the standings can always be rebuilt from the results, scoring rule and tiebreakers, with nothing stored secretly (this

is why stats are not their own classes); only finished contests give points; the scoring rule lives here, not in the sport, so the same sport can run under different scoring.

- **Boundaries:** doesn't decide any contest's outcome (the contest does); doesn't treat the stored leaderboard as the source of truth; doesn't know the inside rules of a contest.

3.2.5 Championship (multi-level competition)

A championship is a competition of competitions: a group of events combined into one overall ranking. **Competition** (one format over one entrant set) is the leaf and **Championship** the composite; both honour a single **Standings** contract (a name plus a computable leaderboard), so a championship is built and queried exactly like a single competition and may nest to any depth.

- **Responsibilities:** holds its child events; owns an **OverallAggregator** (e.g. a medal table) that combines the children's leaderboards into one; produces the overall standing on demand.
- **Invariants:** the overall table is a pure function of the children's decided results and the aggregator, always recomputable, never cached; only decided results contribute.
- **Boundaries:** doesn't run any contest or know a discipline's inside rules; it only aggregates standings its children already produce.

3.3 Strategy contracts

Each contract defers its "how" to a strategy: an interface whose implementation is chosen per discipline or format. This is where the football-versus-chess differences become concrete objects. Two strategies govern a single contest (termination, outcome); four govern a whole competition (scoring, fixtures, standings, tiebreak): six in total. They share one small value type, the **StandingContribution**: a sport-agnostic summary of how one participation did, produced by **Result** and read by **ScoringRule**, carrying a position and/or a win/draw/loss token plus a completed flag, so scoring can award points without knowing the sport.

3.3.1 TerminationRule

Decides when a contest is over.

- **Behaviour:** called repeatedly as the contest progresses; it inspects the current state (elapsed time, a declared condition, or each participation's completion, depending on the discipline) and reports whether the contest may move to a finished state, and which kind.
- **Implementations:** **TimeLimit** (football: 90' + stoppage, optional extra time); **TerminalCondition** (chess: checkmate, resignation, draw conditions).
- **Invariants:** only ever moves toward finished, never backward; answers "is it over?" only, never "who won?".

3.3.2 OutcomeRule

Determines the result of a finished contest.

- **Behaviour:** called once, after termination is confirmed; it reads whatever the discipline uses (event counts, a declared condition, measured values) and constructs the Result, placing every participation in order.
- **Implementations:** EventDerived (football: count goals from the event log); Declared (chess: the terminal condition names the winner or draw). The measured, aggregated and judged modes are realised by the probe disciplines (race, tennis, figure skating) and are not instantiated in the two assigned modules.
- **Invariants:** runs only on a finished contest; covers exactly its participations, none dropped (DNF and DSQ placed last or flagged); deterministic; ties are representable.

Termination and outcome stay separate even when one event answers both: in chess the checkmate ends the game and names the winner, yet the two contracts remain distinct.

3.3.3 ScoringRule

Converts how a participant did into the points it earns, within one competition.

- **Behaviour:** called per participation of each decided contest; it reads the StandingContribution (a token such as “won” or “3rd”) and returns points, never the raw result and never the sport. Knockouts pass a round-reached descriptor instead.
- **Implementations:** WinDrawLoss(3,1,0) (football); WinDrawLoss(1,0.5,0) (chess); PositionPoints(table) (race circuit); RoundReached(table) (tennis tour).
- **Invariants:** owned by the competition, not the sport, so the same sport may score differently elsewhere; a pure function of the StandingContribution alone.

3.3.4 FixtureGenerator

Decides which contests exist, possibly adaptively.

- **Behaviour:** produces the next contests, either the whole schedule upfront or one round at a time; adaptive formats read current standings, seeded formats read entry seedings. Participations come already assigned.
- **Implementations:** RoundRobin (each meets every other, optionally home and away); SwissPairing (paired by current score, avoiding rematches); KnockoutBracket (seeded single-elimination, with byes); SingleContest (all entrants in one contest).
- **Invariants:** only the competition’s entrants appear; no unsanctioned contest; adaptive formats read only decided results.

3.3.5 StandingsAggregator

Builds the standings table from finished contests.

- **Behaviour:** on demand, it applies the ScoringRule to each participation’s StandingContribution, accumulates points per participant, orders them, and hands any level group to the Tiebreak chain. Always recomputed, never cached.

- **Implementations:** PointsTable (sum, for leagues and Swiss); BracketProgression (who advanced, for knockouts); SingleRanking (one contest's order is the standing, for a single race).
- **Invariants:** a pure function of decided results, ScoringRule and Tiebreak, always recomputable; only decided contests contribute.

3.3.6 Tiebreak

Orders participants who are level on the primary standing.

- **Behaviour:** invoked only on a tie; it applies its criterion to the tied group and the decided results, returns an order, and passes any remaining tie to the next criterion in the chain or leaves a genuine tie.
- **Implementations:** GoalDifference (football); HeadToHead (the direct contest); Buchholz / Sonneborn-Berger (chess Swiss); Countback (race circuit); DeclaredTie (no break).
- **Invariants:** invoked only on ties; deterministic; may leave a genuine tie; never changes points; reads only decided results.

Mapped to the two assigned disciplines the framework stays identical and only the plugged-in objects change. **Football** = TimeLimit + EventDerived + WinDrawLoss(3,1,0) + RoundRobin + PointsTable + GoalDifference. **Chess** = TerminalCondition + Declared + WinDrawLoss(1,0.5,0) + SwissPairing + PointsTable + Buchholz.

3.4 Relationships and Multiplicities

Notation: ◆ = owns (composition), → = refers to; 1, 0..1, 1.., 0..* are counts.*

The headline fix is Participation as an association class. A participant takes part in many contests; a contest involves several participants; and the role and outcome belong to the pairing, not to either side. So: Participant 0..* —(Participation)— 1..* Contest. One Participation exists per (participant, contest) pair and carries that pair's role and outcome. This is the concept entirely missing before.

Ownership (composition):

- Competition ◆— 1..* Contest (one for a single race, many for a league).
- Championship ◆— 1..* Standings (its events: single Competitions or nested Championships).
- Contest ◆— 1..* Participation (each contest level has its own participations, even when they point to the same participants).
- Contest ◆— 0..1 Result (none until decided, exactly one after).

References (things that outlive the contest):

- Participation → 1 Participant.
- Contest → 1 TerminationRule, 1 OutcomeRule.
- Competition → 1 Sport (the profile that supplies the default rules).

- Competition → 1 ScoringRule, 1 FixtureGenerator, 1 StandingsAggregator, 0..* Tiebreak (ordered, applied in sequence).
- Competition → 1..* Participant (its entrants; a participant can enter many competitions; seeding, if any, lives on this entry).
- Championship → 1 OverallAggregator (how the events combine, e.g. a medal table).

Derived, never stored as truth:

- Result yields one StandingContribution per Participation (what ScoringRule reads).
- StandingsAggregator computes the leaderboard from decided results, ScoringRule and Tiebreak, which is always recomputable, so there are no stored Stats classes.

3.5 Lifecycle and States

Two levels, because the marathon probe showed that the contest finishing and an individual finishing are different questions.

Contest state machine:

- Normal path: Scheduled → InProgress → Finished.
- Pause: InProgress ↔ Suspended (chess adjournment, weather hold; resuming is allowed, this is a pause, not a reversal).
- Ended without normal completion, from Scheduled or InProgress: Forfeited, Walkover, Abandoned, Cancelled/Void.
- Correction only: Finished → Reopened → InProgress, explicit and audited, never a silent edit.

Decided states (those that carry a Result): Finished, Forfeited and Walkover. An abandoned contest carries no result on its own: the discipline decides whether it is void, replayed, or stands, and if it stands it is *explicitly accepted*, which moves it to Finished. Scheduled, InProgress, Suspended, Abandoned and Cancelled/Void carry no Result.

State-bound rules:

- A Result exists if and only if the contest is decided.
- Only decided contests feed the standings; Cancelled/Void contribute nothing.
- A nested contest can become decided only once its TerminationRule is satisfied given its children.

Participation status (the second level):

- Entered → Active → Completed / DidNotFinish / Disqualified, plus DidNotStart (set at kick-off) and Withdrew (before start).
- A participation reaches a terminal status only at or after the contest becomes decided: in a race runner by runner, in football both sides at full time.
- For ranking disciplines the contest reaches Finished only when every participation is terminal or the cutoff passes.

4. Decision Flows, Differences as Design Decisions, and the Diagram

The previous three sections diagnosed the old model, broke it with probes, and rebuilt it on precise definitions. This section records the work done in resolving the design: the small number of event flows that actually carry a decision, the reading of each real-world difference as a concrete modelling consequence, and the class diagram itself, split into core and extension. It deliberately omits the fourteen CRUD use cases and the self-evident sequence diagrams of the first submission; they describe generic application plumbing and add nothing to the domain model. What follows keeps only what a reader could not have guessed from the class list alone.

4.1 Narration of the Non-Trivial Flows

Three flows are worth narrating because each forces a choice the static diagram cannot show on its own: where termination ends and outcome begins, how results of entirely different shapes still feed one leaderboard, and how an adaptive round is paired and then ranked. Every other flow in the system is either a trivial read or a create/update that the diagram already implies.

4.1.1 How a chess match resolves termination and outcome

The decision this flow exposes is that termination and outcome are two contracts, not one, even though in chess a single event answers both. The naive model folds them together: “the game ends when someone wins.” The probe-hardened model keeps them apart, and the flow shows why.

A chess Contest is configured with a **TerminalCondition** termination rule and a **Declared** outcome rule. While the game is **InProgress**, moves arrive and are appended to the contest’s optional event log. After each move the contest asks its **TerminationRule** a single question: may I move to a finished state, and which kind? For most moves the answer is no. When a move produces checkmate, the rule answers yes, finished by terminal condition; on a resignation it answers yes, finished by resignation; on a threefold-repetition or stalemate claim it answers yes, finished as a draw. The termination rule names the kind of ending but never names the winner.

Only once the contest has transitioned to **Finished** does it call its **OutcomeRule**. The **Declared** rule reads the terminal condition that ended the game and constructs the **Result**: for a checkmate or resignation it orders the two participations winner-first and writes each participation’s standing-contribution (a “won” and a “lost” token); for a draw it ranks them equal and writes two “drew” tokens. The result exists exactly once, is attached to the now-finished contest, and never changes afterwards.

The decision made visible here is the separation itself. A checkmate is one real-world event that both ends the game and decides it, yet the model routes it through two distinct

contracts in sequence: terminate, then determine. Keeping them separate is what lets a football match, where a referee's whistle ends the match by time but the goals tally decides it, and a marathon, where a cutoff ends the contest but measured times decide it, reuse the identical Contest object with different rules plugged in. Folding the two together, as the first model did, is precisely what made the old design unable to host any discipline beyond the one it was written for.

An adjournment is the second subtlety this flow carries. If play is suspended, the contest moves InProgress → Suspended and, crucially, no Result is produced, because a result exists if and only if the contest is decided. On resumption it returns to InProgress; this is a pause, not a reversal, and the finished-state invariant is never touched.

4.1.2 How heterogeneous results feed one leaderboard

This flow exposes the decision that the leaderboard must read a sport-agnostic token, never the raw result. A league of football matches, a chess Swiss, and a marathon produce results of utterly different shapes — a 2–1 scoreline, a win/draw/loss symbol, a list of finishing times — yet all three must become points in one standings table. The model achieves this by inserting one small value type between the result and the scoring.

When a Contest finishes, its Result produces, for each participation, a single **StandingContribution**: a position and/or a win/draw/loss token plus a completed flag. This is the only thing the result hands outward. The contribution carries no goals, no times, no board state — nothing that betrays which sport produced it.

The Competition's StandingsAggregator then walks every decided contest and, for each participation, passes its StandingContribution to the competition's ScoringRule. The scoring rule is a pure function of that token alone: **WinDrawLoss(3,1,0)** turns a football "won" into three points; **WinDrawLoss(1,0.5,0)** turns a chess "won" into one; **PositionPoints(table)** turns a marathon "3rd" into whatever that circuit awards for third. The aggregator accumulates the points per participant, orders them, and hands any level group to the tiebreak chain.

The decision laid bare is that the leaderboard never sees a goal. The same football Result object, scored under a different ScoringRule in a different competition, yields different points without any change to the result, the contest, or the sport. This is the direct structural answer to the old model's mistake of attaching the points system to the sport: here the points live on the competition, and the result speaks only in standing-contributions. Heterogeneous results converge precisely because they are forced through one narrow, sport-blind channel.

4.1.3 How a Swiss or league round computes standings with tiebreakers

The decision in this flow is that some formats must read the current standings before they can generate the next contests, so fixture generation and standings aggregation are coupled in a loop — and that ranking is a two-stage process, points first, tiebreak second. A league hides the loop because its whole schedule is fixed up front; a Swiss makes it unavoidable.

For a league the FixtureGenerator is **RoundRobin**: it emits the entire schedule at the start, every entrant against every other, optionally home and away. The competition plays the contests in any order; standings are merely recomputed on demand. For a Swiss the generator is **SwissPairing**, and it cannot emit round two until round one is decided: it reads the current standings, pairs players on equal or near-equal scores, avoids rematches, and only then produces the next contests. The adaptive format reads only decided results, never provisional ones.

Ranking, in both cases, is computed and never stored. On demand the StandingsAggregator (a **PointsTable** for both league and Swiss) applies the ScoringRule to every decided participation's standing-contribution, sums the points per participant, and orders them. Where participants are level on points, and only there, it invokes the Tiebreak chain: football applies **GoalDifference** then head-to-head; a chess Swiss applies **Buchholz** or Sonneborn-Berger. Each criterion takes the tied group, returns an order, and passes any still-tied remainder to the next criterion, or leaves a genuine tie. A tiebreak never changes points and reads only decided results.

The decision made visible is that the standings table is derived, not authoritative. It can always be rebuilt from the decided results, the scoring rule and the tiebreak chain, with nothing cached. This is exactly why the redesigned model has no stored statistics classes: a FootballStats or ChessStats subclass would be a cached view masquerading as a source of truth. The Swiss loop relies on this guarantee — if standings could drift from the results, adaptive pairing would pair on a lie.

4.2 Differences as Design Decisions

The first submission listed real differences between football and chess in a comparison table but never said what any of them meant for the model. The teacher's feedback named this precisely: differences described but not interpreted. This section repairs that gap by rewriting each difference as a chain from observation to mechanism, in the form difference $\Rightarrow$ consequence for the model $\Rightarrow$ resulting rule. Each one is a design decision the diagram then encodes.

- **Numeric versus symbolic result:** football produces a numeric scoreline (2–1) while chess produces a symbol (win/draw/loss) $\Rightarrow$ no single concrete result type can hold both, so Result must be an interface with many shapes behind it, and the leaderboard must not read either shape directly $\Rightarrow$ Result exposes only an ordering plus a per-participation StandingContribution; the leaderboard reads points via the ScoringRule, never goals and never the win symbol.
- **Result computed versus result declared:** football's result is derived by counting goals from the event log, whereas chess's result is declared the instant a terminal condition occurs $\Rightarrow$ the link from events to result cannot be assumed; one discipline computes, the other declares, so outcome determination must be a pluggable strategy rather than fixed behaviour on the match $\Rightarrow$ OutcomeRule has an EventDerived implementation for

football and a Declared implementation for chess; the event log is an optional capability the result never assumes.

- **Ends by clock versus ends by condition:** a football match ends when a fixed time elapses; a chess game ends when a board condition (mate, resignation, draw) is reached $\Rightarrow$ termination cannot be a single attribute like a duration; what ends a contest varies by discipline and must be asked, not stored $\Rightarrow$ TerminationRule is a strategy with TimeLimit for football and TerminalCondition for chess, queried repeatedly and only ever moving toward finished.
- **Points system belongs to the tournament, not the sport:** football commonly awards 3/1/0 while chess awards 1/0.5/0, yet the same sport can be scored differently in different competitions $\Rightarrow$ a points system attached to the sport is misplaced; it is a property of the competition's format, so the same sport object must be reusable under different scoring $\Rightarrow$ ScoringRule is owned by the Competition, not the Sport; WinDrawLoss(3,1,0) and WinDrawLoss(1,0.5,0) are competition-level choices.
- **Fixtures fixed versus fixtures adaptive:** a football league fixes its whole schedule up front, while a chess Swiss pairs each new round from the current standings $\Rightarrow$ fixture generation cannot be a one-shot list; some formats must read decided results between rounds, so generation must be able to run round by round $\Rightarrow$ FixtureGenerator is a strategy: RoundRobin emits everything at once, SwissPairing emits one round at a time after reading the standings.
- **Different tiebreakers:** football breaks ties by goal difference and head-to-head; chess Swiss breaks them by Buchholz or Sonneborn-Berger $\Rightarrow$ tiebreaking cannot be a fixed comparator; the criteria differ per discipline and must be chainable and replaceable $\Rightarrow$ Tiebreak is an ordered chain of strategies owned by the Competition, invoked only on a level group and free to leave a genuine tie.
- **Event log authoritative versus event log a mere record:** in football the event log of goals is the source from which the result is computed; in chess the move log is only a record and the result is declared independently $\Rightarrow$ the event log cannot be a universal mechanism that decides results; its authority varies, so it must be an optional capability rather than a core assumption $\Rightarrow$ EventRecording is an optional capability; the result is built by the OutcomeRule, which may or may not read events depending on the discipline.
- **One opaque side versus the same opaque side:** a football side is a team of eleven; a chess side is a single player; both nonetheless enter, take part, and finish identically $\Rightarrow$ the difference between a team and an individual must not leak into the core; a participant has to be treated as one opaque competing thing $\Rightarrow$ Participant is a single opaque entity.

Read together, these decisions explain why the redesigned model has no football or chess subclasses of Contest, Result or Stats. Every difference that the first model would have expressed by subclassing is instead expressed by plugging a different strategy into one

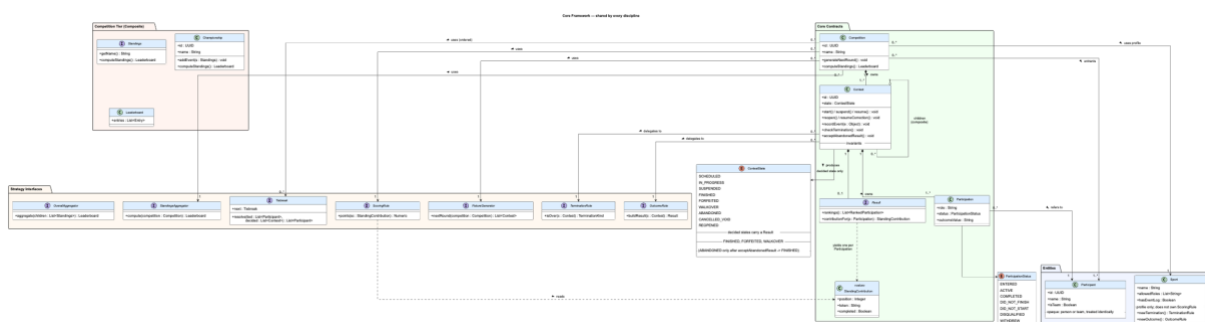
configurable contest and one configurable competition. The two disciplines reduce to two bundles of strategy objects: football is **TimeLimit** + **EventDerived** + **WinDrawLoss(3,1,0)** + **RoundRobin** + **PointsTable** + **GoalDifference**; chess is **TerminalCondition** + **Declared** + **WinDrawLoss(1,0.5,0)** + **SwissPairing** + **PointsTable** + **Buchholz**. The framework code is identical across both; only the plugged-in objects change.

4.3 The Class Diagram

With concepts, mechanisms and relationships settled, the diagram can finally be drawn. It is presented in two halves, because the design's central claim is that the core is closed and the disciplines are additive: the core framework holds everything shared by every imaginable discipline, and each extension module contributes only concrete strategy implementations that plug into it. A single diagram drawn before the probes is what produced the flat model the first section criticised; this one is drawn last, on purpose.

4.3.1 Core framework

The core contains the two entities that survived every probe, Participant and Sport; the four universal contracts Competition, Contest, Participation and Result; and the six strategy interfaces they defer to. Nothing here is specific to football or chess. The relationships are the ones fixed in the definitions: Participation is the association class on the Participant–Contest line, carrying the role and outcome that belong to the pairing; Competition composes one-or-more Contests, each Contest composes one-or-more Participations and at most one Result; and the strategies are referenced, never owned, so they can be swapped per discipline. Contest and Competition are concrete and configured by strategies rather than subclassed; only Result is a true many-shapes interface. The same diagram carries the competition tier's Composite: Competition (leaf) and Championship (composite) both realise Standings, so a single competition and a whole multi-level championship are built and queried uniformly — this is where the Composition axis is realised, with the Contest staying atomic.

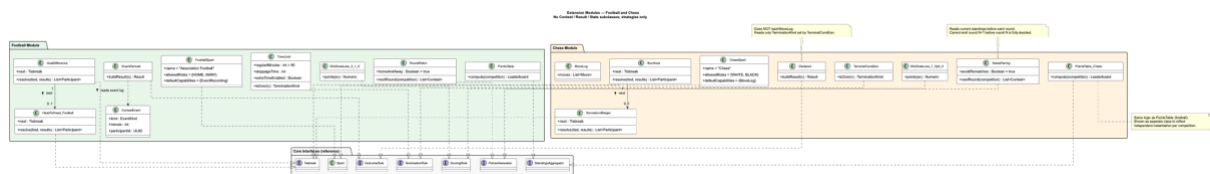


4.3.2 Football and chess extension modules

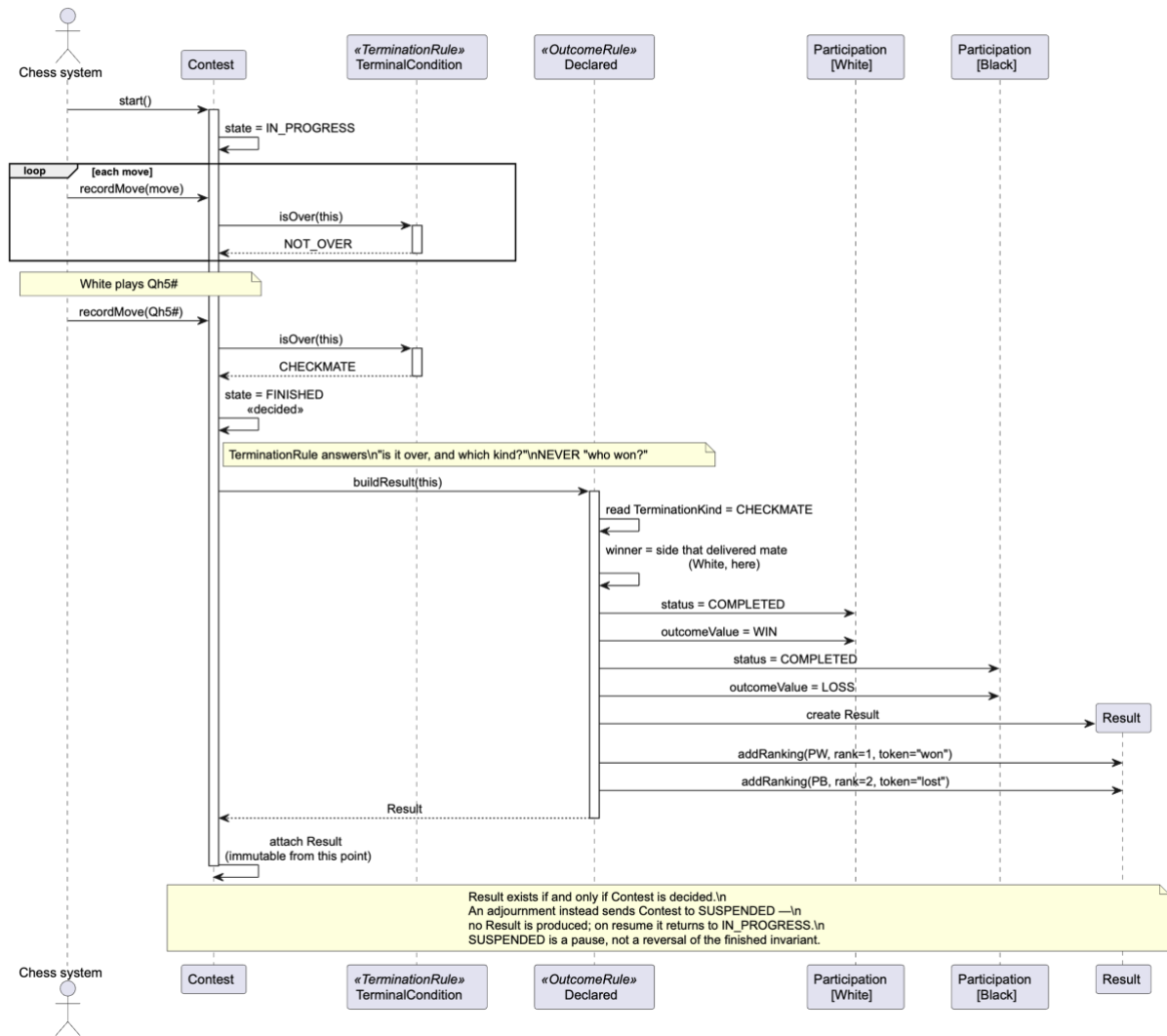
Because the core is closed, the extension modules introduce no new Contest, Result or Stats subclasses; that absence is the design argument, not an omission, since those parallel hierarchies were the smell the redesign set out to remove. Each module supplies concrete

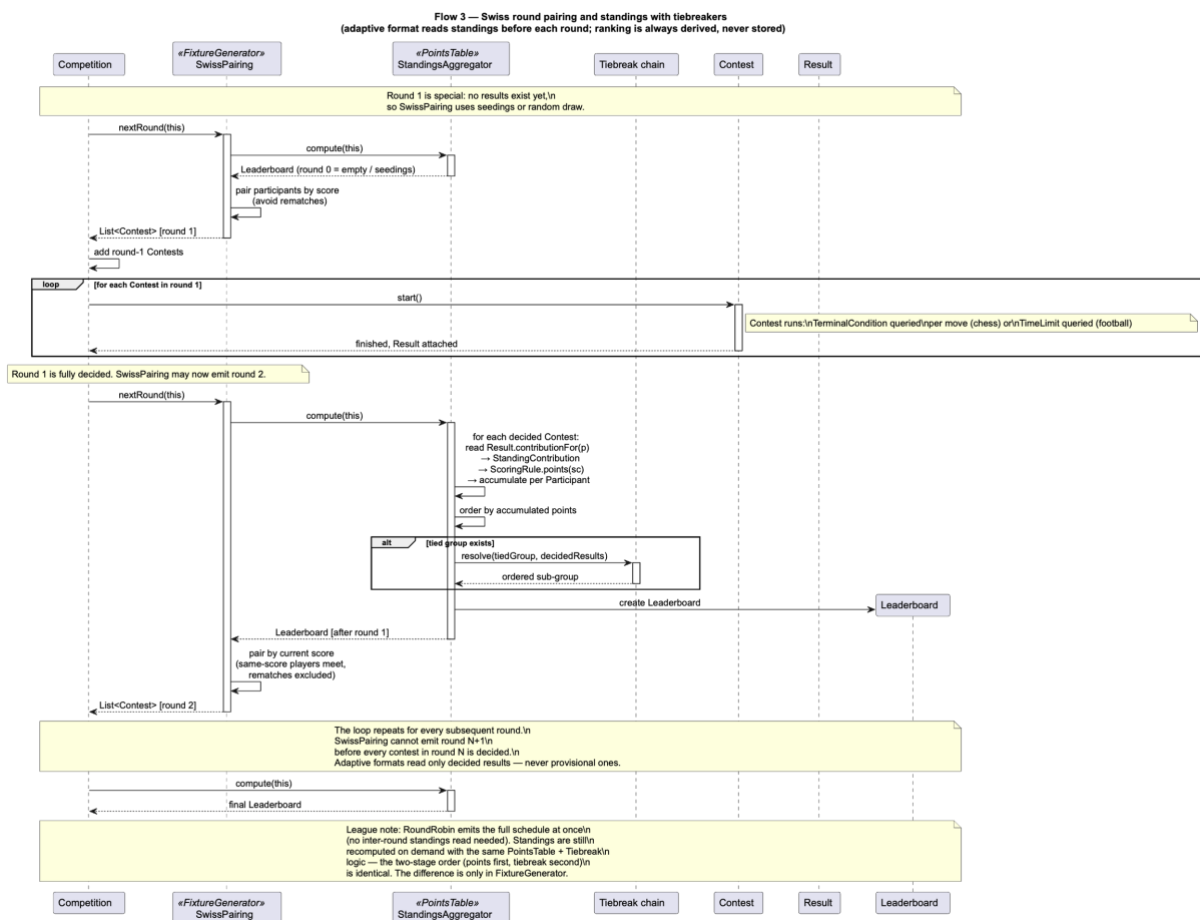
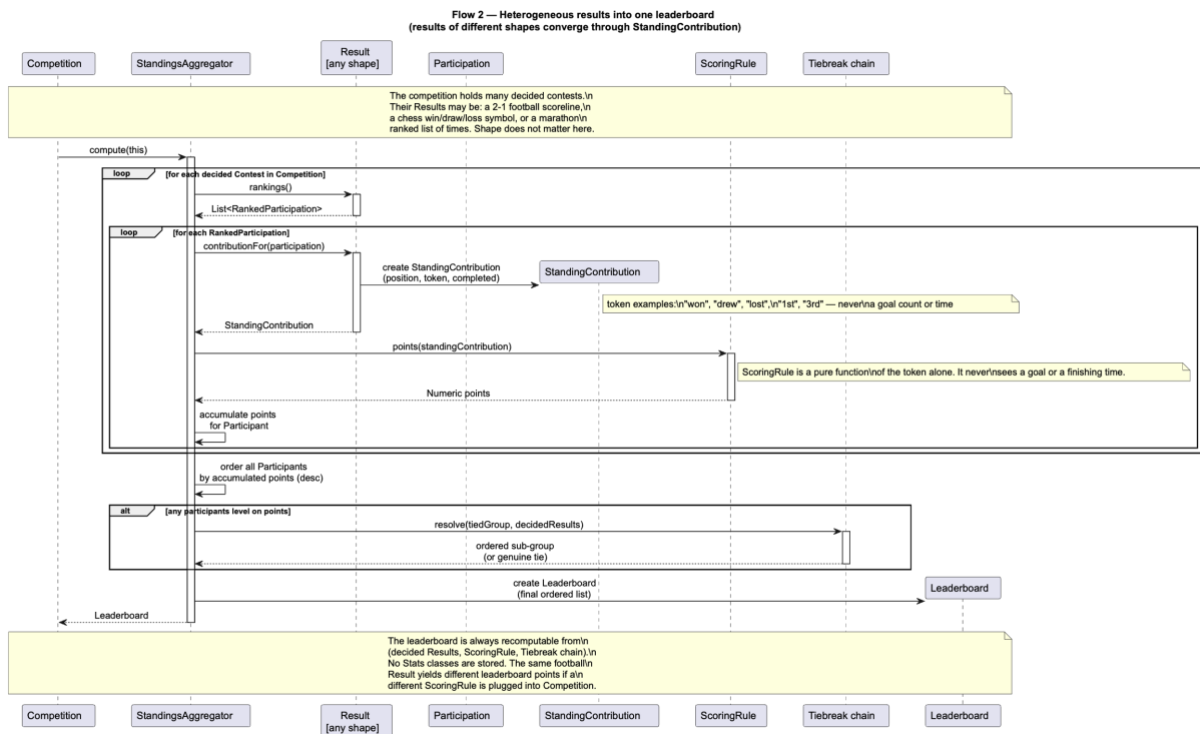
classes that realise the core strategy interfaces, plus a discipline profile. Football contributes **TimeLimit**, **EventDerived**, **WinDrawLoss(3,1,0)**, **RoundRobin**, **PointsTable** and **GoalDifference**; chess contributes **TerminalCondition**, **Declared**, **WinDrawLoss(1,0.5,0)**, **SwissPairing**, **PointsTable** and **Buchholz**. A football competition is the core objects with the football bundle injected; a chess competition is the same core objects with the chess bundle injected.

The marathon, tennis and figure-skating probes of Section 2 were stress tests, not deliverables; only the two assigned disciplines are realised as modules here.

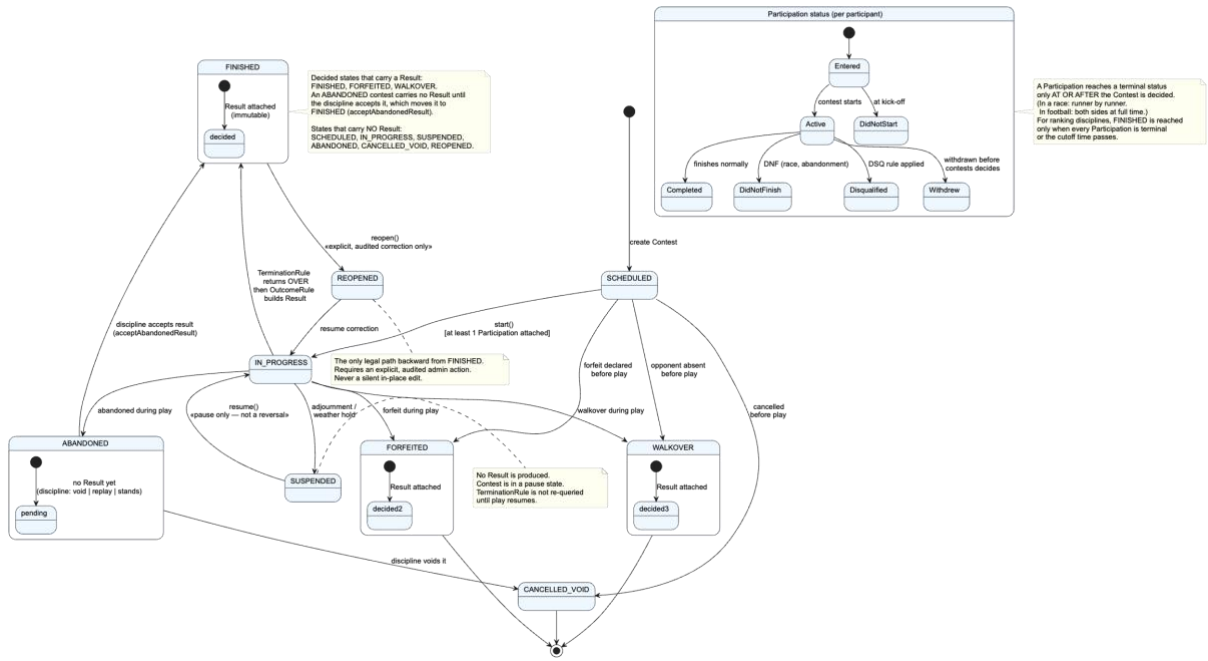


Flow 1 — Chess match: termination then outcome
(the two contracts stay separate even when one event answers both)





Contest State Machine
(section 3.5 — two separate levels: contest state and participation status)



5. Conclusion

This report set out to correct a model that, by both the teacher's account and our own, had failed in a specific and instructive way: it named its concepts without defining them, left their relationships unstated, and silently assumed that every contest in the world looks like football or chess. The correction was not a matter of adding classes but of finding the assumptions that were wrongly hard-wired and replacing each with an explicit choice.

The method that exposed those assumptions was to break the model on purpose. Three probes each attacked a different load-bearing belief. What survived every probe was small: only Participant and Sport stood untouched. Everything else resolved into four universal contracts, a layer of pluggable strategies, and a set of optional capabilities, with no core inheritance and the Composite pattern as the only structural nesting.

From that settled design the deliverables the first submission lacked finally followed. Every concept now carries a definition, responsibilities and invariants. Relationships and multiplicities are stated, with Participation restored as the association class that the original sequence diagrams had used but never modelled. The real-world differences between the assigned disciplines are read as design decisions rather than merely tabulated, each difference traced to the interface, strategy or capability it forces. The flows worth narrating are reduced to the three that carry a genuine decision, and the class diagram is split into a closed core and two additive extension modules.

The result is a framework whose central property is that adding a discipline means writing strategies, not editing the core. Football and chess differ only by the bundles of objects plugged into one configurable contest and one configurable competition. The design is therefore flexible in the precise sense the brief demanded: the contracts shared by all imaginable disciplines are carried in the core, the responsibilities a given discipline does not use simply go unplugged, and no future sport requires the framework itself to change.